CO3-ASSESSMENT TOOL -2

CSA-0603-DESIGN AND ANALYSIS OF ALGORITHMS

K Sravan Kumar- 192311210 **Date:5-08-2026**

TITLE

Web Server Log Analysis Using Sorting and Binary Search (Divide and Conquer Approach)

Description of the Real-World Problem

Web servers generate thousands or even millions of log entries every day. These logs contain important information such as IP addresses, timestamps, request methods, URLs, response codes, and response times. Searching for a particular log entry directly from an unsorted log file is slow and inefficient. Therefore, sorting the log records first and then applying Binary Search helps in retrieving required information quickly and efficiently.

Problem Statement

Design a system that efficiently processes a large web server log file by:

- Sorting the log records based on a selected field (Timestamp or IP Address).
- Searching for a specific log entry using Binary Search.
- Reducing search time for large datasets.
- Applying Divide and Conquer techniques to improve efficiency.
- Analyzing the overall time complexity.

Algorithm Used

Sorting Algorithm: Merge Sort

- Divide the log file into two halves.
- Recursively sort each half.
- Merge the sorted halves.

Searching Algorithm: Binary Search

- Compare the middle element with the target.
- If equal, return the result.
- If smaller, search the right half.
- Otherwise, search the left half.

Both algorithms follow the **Divide and Conquer** technique.

Justification for Choosing the Algorithm (Why This Algorithm?)

- Merge Sort guarantees **O(n log n)** time complexity.
- Efficient for very large datasets.
- Stable sorting algorithm.
- Binary Search provides **O(log n)** search time.
- Suitable for sorted log files.
- Combination significantly reduces processing time.

Complexity Analysis

Operation	**Algorithm**	**Best Case**	**Average Case**	**Worst Case**	
Sorting	Merge Sort	O(n log n)	O(n log n)	O(n log n)	
Searching	Binary Search	O(1)	O(log n)	O(log n)	
Overall	Merge Sort + Binary Search	O(n log n)	O(n log n)	O(n log n)	

Space Complexity

- Merge Sort: **O(n)**
- Binary Search: **O(1)** (Iterative)

Manual Execution (Step-by-Step Process)

Sample Log IDs

404

201

503

301

102

450

Step 1: Original Logs

404

201

503

301

102

450

Step 2: Divide

404 201 503

301 102 450

Step 3: Further Divide

404

201

503

301

102

450

Step 4: Merge in Sorted Order

201 404

201 404 503

102 301

102 301 450

Step 5: Final Sorted List

102

201

301

404

450

503

Step 6: Binary Search for 404

Initial List

102 201 301 404 450 503

Middle = 301

404 > 301

Search Right Half

Right Half

404 450 503

Middle = 450

404 < 450

Search Left Half

Remaining

404

Element Found.

Applications

- Web server log management
- Network traffic monitoring
- Cybersecurity analysis
- Cloud server monitoring
- Database indexing
- System performance analysis

Advantages

- Fast searching after sorting.
- Suitable for large datasets.
- Efficient Divide and Conquer approach.
- Predictable time complexity.
- Stable sorting with Merge Sort.
- Reduces overall processing time.

Limitations

- Merge Sort requires additional memory.
- Binary Search only works on sorted data.
- Initial sorting takes time.
- Not ideal for frequently changing datasets.
- Large log files require more storage.
- Merge Sort has recursive overhead.

Comparison with Other Algorithms

Algorithm	**Time Complexity**	**Suitable for Large Logs**	**Stable**	**Divide & Conquer**
Bubble Sort	$O(n^2)$	No	Yes	No
Selection Sort	$O(n^2)$	No	No	No
Insertion Sort	$O(n^2)$	Moderate	Yes	No
Quick Sort	O(n log n) Average	Yes	No	Yes
Merge Sort	**O(n log n)**	**Yes**	**Yes**	**Yes**
Linear Search	O(n)	Slow	—	No
Binary Search	**O(log n)**	**Very Fast**	—	**Yes**

Program (Python)

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)
```

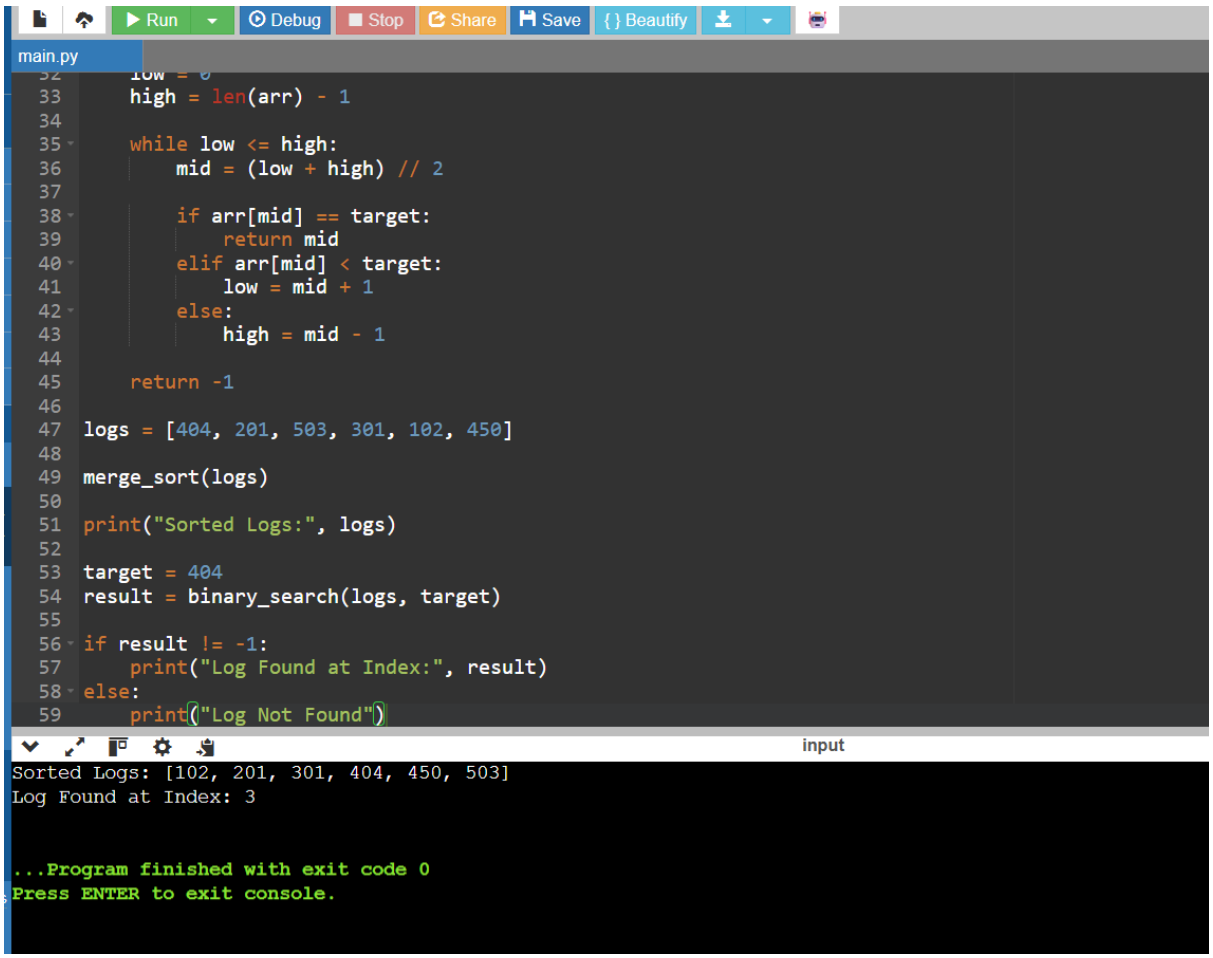
```
    i = j = k = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1

    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1

    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1

def binary_search(arr, target):
    low = 0
    high = len(arr) - 1
```

```
    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

logs = [404, 201, 503, 301, 102, 450]

merge_sort(logs)

print("Sorted Logs:", logs)

target = 404
result = binary_search(logs, target)

if result != -1:
    print("Log Found at Index:", result)
else:
    print("Log Not Found")
```

Screenshot of the Program

Insert a screenshot of the above Python code from your IDE (VS Code, PyCharm, IDLE, or Jupyter Notebook).

Output Screenshot



Conclusion

Sorting and Binary Search together provide an efficient solution for analyzing large web server log files. Merge Sort organizes log records in **O(n log n)** time, while Binary Search retrieves required log entries in **O(log n)** time. The Divide and Conquer approach improves scalability, reduces search time, and makes the system suitable for handling large volumes of server logs in real-world applications. This combination is widely used in log analysis, database indexing, and network monitoring systems due to its efficiency and reliability.